

DNS

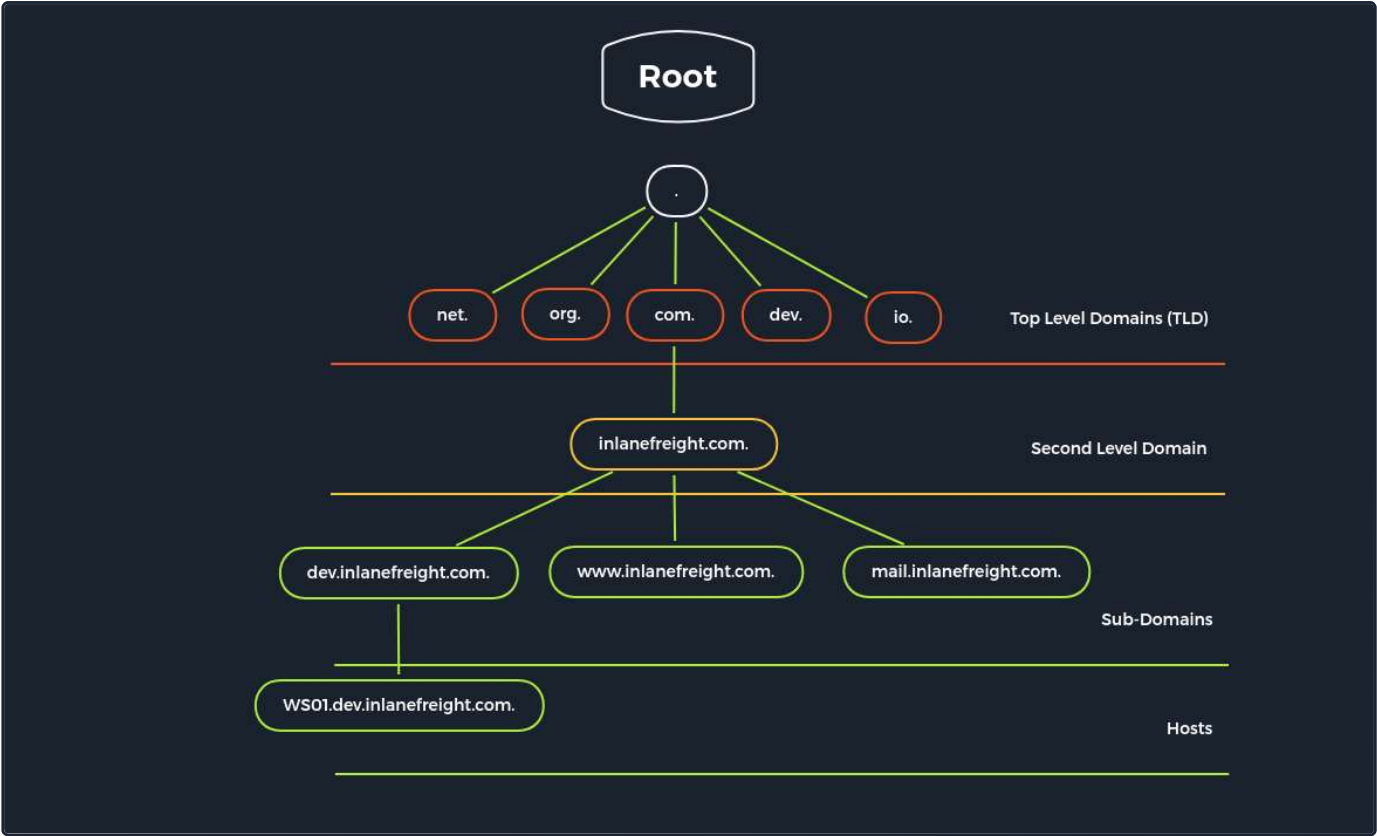
Domain Name System (DNS) is an integral part of the Internet. For example, through domain names, such as academy.hackthebox.com or www.hackthebox.com, we can reach the web servers that the hosting provider has assigned one or more specific IP addresses. DNS is a system for resolving computer names into IP addresses, and it does not have a central database. Simplified, we can imagine it like a library with many different phone books. The information is distributed over many thousands of name servers. Globally distributed DNS servers translate domain names into IP addresses and thus control which server a user can reach via a particular domain. There are several types of DNS servers that are used worldwide:

- DNS root server
- Authoritative name server
- Non-authoritative name server
- Caching server
- Forwarding server
- Resolver

Server Type	Description
DNS Root Server	The root servers of the DNS are responsible for the top-level domains (TLD). As the last instance, they are only requested if the name server does not respond. Thus, a root server is a central interface between users and content on the Internet, as it links domain and IP address. The Internet Corporation for Assigned Names and Numbers (ICANN) coordinates the work of the root name servers. There are 13 such root servers around the globe.
Authoritative Nameserver	Authoritative name servers hold authority for a particular zone. They only answer queries from their area of responsibility, and their information is binding. If an authoritative name server cannot answer a client's query, the root name server takes over at that point.
Non-authoritative Nameserver	Non-authoritative name servers are not responsible for a particular DNS zone. Instead, they collect information on specific DNS zones themselves, which is done using recursive or iterative DNS querying.
Caching DNS Server	Caching DNS servers cache information from other name servers for a specified period. The authoritative name server determines the duration of this storage.
Forwarding Server	Forwarding servers perform only one function: they forward DNS queries to another DNS server.
Resolver	Resolvers are not authoritative DNS servers but perform name resolution locally in the computer or router.

DNS is mainly unencrypted. Devices on the local WLAN and Internet providers can therefore hack in and spy on DNS queries. Since this poses a privacy risk, there are now some solutions for DNS encryption. By default, IT security professionals apply **DNS over TLS (DoT)** or **DNS over HTTPS (DoH)** here. In addition, the network protocol **DNSCrypt** also encrypts the traffic between the computer and the name server.

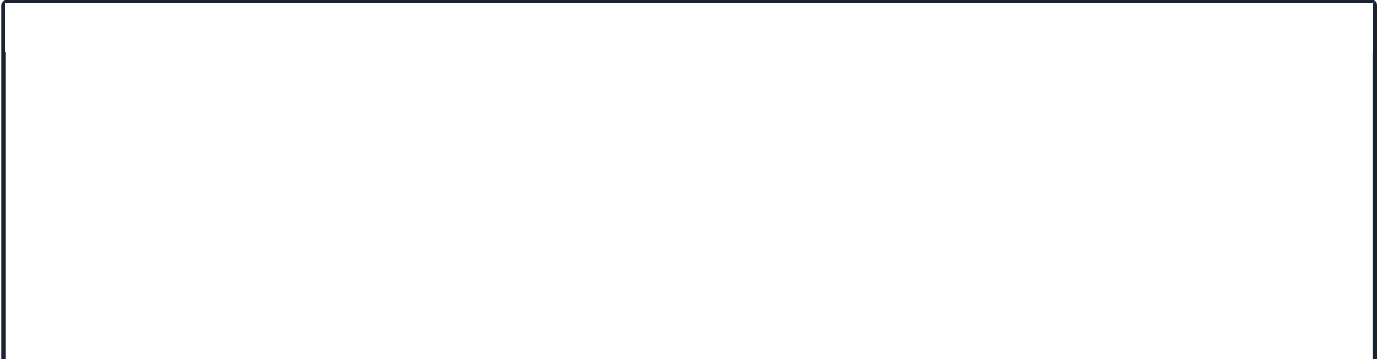
However, the DNS does not only link computer names and IP addresses. It also stores and outputs additional information about the services associated with a domain. A DNS query can therefore also be used, for example, to determine which computer serves as the e-mail server for the domain in question or what the domain's name servers are called.



Different **DNS records** are used for the DNS queries, which all have various tasks. Moreover, separate entries exist for different functions since we can set up mail servers and other servers for a domain.

DNS Record	Description
A	Returns an IPv4 address of the requested domain as a result.
AAAA	Returns an IPv6 address of the requested domain.
MX	Returns the responsible mail servers as a result.
NS	Returns the DNS servers (nameservers) of the domain.
TXT	This record can contain various information. The all-rounder can be used, e.g., to validate the Google Search Console or validate SSL certificates. In addition, SPF and DMARC entries are set to validate mail traffic and protect it from spam.
CNAME	This record serves as an alias. If the domain www.hackthebox.eu should point to the same IP, and we create an A record for one and a CNAME record for the other.
PTR	The PTR record works the other way around (reverse lookup). It converts IP addresses into valid domain names.
SOA	Provides information about the corresponding DNS zone and email address of the administrative contact.

The **SOA** record is located in a domain's zone file and specifies who is responsible for the operation of the domain and how DNS information for the domain is managed.



```
dbcyph0n@htb[/htb]$ dig soa www.inlanefreight.com

; <<>> DiG 9.16.27-Debian <<>> soa www.inlanefreight.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 15876
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;www.inlanefreight.com.      IN      SOA

;; AUTHORITY SECTION:
inlanefreight.com.      900      IN      SOA      ns-161.awsdns-20.com. awsdns-hostmaster.amazon.com. 1 7200 900 12

;; Query time: 16 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Thu Jan 05 12:56:10 GMT 2023
;; MSG SIZE rcvd: 128
```

The dot (.) is replaced by an at sign (@) in the email address. In this example, the email address of the administrator is `awsdns-hostmaster@amazon.com`.

Default Configuration

There are many different configuration types for DNS. Therefore, we will only discuss the most important ones to illustrate better the functional principle from an administrative point of view. All DNS servers work with three different types of configuration files:

1. local DNS configuration files
2. zone files
3. reverse name resolution files

The DNS server `Bind9` is very often used on Linux-based distributions. Its local configuration file (`named.conf`) is roughly divided into two sections, firstly the options section for general settings and secondly the zone entries for the individual domains. The local configuration files are usually:

- `named.conf.local`
- `named.conf.options`
- `named.conf.log`

It contains the associated RFC where we can customize the server to our needs and our domain structure with the individual zones for different domains. The configuration file `named.conf` is divided into several options that control the behavior of the name server. A distinction is made between `global options` and `zone options`.

Global options are general and affect all zones. A zone option only affects the zone to which it is assigned. Options not listed in `named.conf` have default values. If an option is both global and zone-specific, then the zone option takes precedence.

Local DNS Configuration

Local DNS Configuration

```
root@bind9:~# cat /etc/bind/named.conf.local

//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";
zone "domain.com" {
    type master;
    file "/etc/bind/db.domain.com";
    allow-update { key rndc-key; };
};
```

In this file, we can define the different zones. These zones are divided into individual files, which in most cases are mainly intended for one domain only. Exceptions are ISP and public DNS servers. In addition, many different options extend or reduce the functionality. We can look these up on the [documentation](#) of Bind9.

A **zone file** is a text file that describes a DNS zone with the BIND file format. In other words it is a point of delegation in the DNS tree. The BIND file format is the industry-preferred zone file format and is now well established in DNS server software. A zone file describes a zone completely. There must be precisely one **SOA** record and at least one **NS** record. The SOA resource record is usually located at the beginning of a zone file. The main goal of these global rules is to improve the readability of zone files. A syntax error usually results in the entire zone file being considered unusable. The name server behaves similarly as if this zone did not exist. It responds to DNS queries with a **SERVFAIL** error message.

In short, here, all **forward records** are entered according to the BIND format. This allows the DNS server to identify which domain, hostname, and role the IP addresses belong to. In simple terms, this is the phone book where the DNS server looks up the addresses for the domains it is searching for.

Zone Files

Zone Files

```

root@bind9:~# cat /etc/bind/db.domain.com

;
; BIND reverse data file for local loopback interface
;
$ORIGIN domain.com
$TTL 86400
@      IN      SOA      dns1.domain.com.      hostmaster.domain.com. (
                                2001062501 ; serial
                                21600       ; refresh after 6 hours
                                3600        ; retry after 1 hour
                                604800     ; expire after 1 week
                                86400 )    ; minimum TTL of 1 day

                                IN      NS      ns1.domain.com.
                                IN      NS      ns2.domain.com.

                                IN      MX      10      mx.domain.com.
                                IN      MX      20      mx2.domain.com.

                                IN      A        10.129.14.5

server1      IN      A        10.129.14.5
server2      IN      A        10.129.14.7
ns1          IN      A        10.129.14.2
ns2          IN      A        10.129.14.3

ftp          IN      CNAME     server1
mx           IN      CNAME     server1
mx2          IN      CNAME     server2
www          IN      CNAME     server2

```

For the IP address to be resolved from the **Fully Qualified Domain Name (FQDN)**, the DNS server must have a reverse lookup file. In this file, the computer name (FQDN) is assigned to the last octet of an IP address, which corresponds to the respective host, using a **PTR** record. The PTR records are responsible for the reverse translation of IP addresses into names, as we have already seen in the above table.

Reverse Name Resolution Zone Files

Reverse Name Resolution Zone Files

```

root@bind9:~# cat /etc/bind/db.10.129.14

;
; BIND reverse data file for local loopback interface
;
$ORIGIN 14.129.10.in-addr.arpa
$TTL 86400
@      IN      SOA      dns1.domain.com.      hostmaster.domain.com. (
                                2001062501 ; serial
                                21600       ; refresh after 6 hours
                                3600        ; retry after 1 hour
                                604800     ; expire after 1 week
                                86400 )    ; minimum TTL of 1 day

                                IN      NS      ns1.domain.com.
                                IN      NS      ns2.domain.com.

5      IN      PTR      server1.domain.com.
7      IN      MX      mx.domain.com.
...SNIP...

```

Dangerous Settings

There are many ways in which a DNS server can be attacked. For example, a list of vulnerabilities targeting the BIND9 server can be found at [CVEdetails](#). In addition, SecurityTrails provides a short [list](#) of the most popular attacks on DNS servers.

Some of the settings we can see below lead to these vulnerabilities, among others. Because DNS can get very complicated and it is very easy for errors to creep into this service, forcing an administrator to work around the problem until they find an exact solution. This often leads to elements being released so that parts of the infrastructure function as planned and desired. In such cases, functionality has a higher priority than security, which leads to misconfigurations and vulnerabilities.

Option	Description
<code>allow-query</code>	Defines which hosts are allowed to send requests to the DNS server.
<code>allow-recursion</code>	Defines which hosts are allowed to send recursive requests to the DNS server.
<code>allow-transfer</code>	Defines which hosts are allowed to receive zone transfers from the DNS server.
<code>zone-statistics</code>	Collects statistical data of zones.

Footprinting the Service

The footprinting at DNS servers is done as a result of the requests we send. So, first of all, the DNS server can be queried as to which other name servers are known. We do this using the NS record and the specification of the DNS server we want to query using the `@` character. This is because if there are other DNS servers, we can also use them and query the records. However, other DNS servers may be configured differently and, in addition, may be permanent for other zones.

DIG - NS Query

DIG - NS Query
<pre>dbcyp0n@htb[/htb]\$ dig ns inlanefreight.htb @10.129.14.128 ; <>> DiG 9.16.1-Ubuntu <>> ns inlanefreight.htb @10.129.14.128 ;; global options: +cmd ;; Got answer: ;; ->>HEADER<- opcode: QUERY, status: NOERROR, id: 45010 ;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 2 ;; OPT PSEUDOSECTION: ; EDNS: version: 0, flags:; udp: 4096 ; COOKIE: ce4d8681b32abaea0100000061475f73842c401c391690c7 (good) ;; QUESTION SECTION: ;inlanefreight.htb. IN NS ;; ANSWER SECTION: inlanefreight.htb. 604800 IN NS ns.inlanefreight.htb. ;; ADDITIONAL SECTION: ns.inlanefreight.htb. 604800 IN A 10.129.34.136 ;; Query time: 0 msec ;; SERVER: 10.129.14.128#53(10.129.14.128) ;; WHEN: So Sep 19 18:04:03 CEST 2021 ;; MSG SIZE rcvd: 107</pre>

Sometimes it is also possible to query a DNS server's version using a class CHAOS query and type TXT. However, this entry must exist on the DNS server. For this, we could use the following command:

DIG - Version Query

DIG - Version Query

```
dbcypb0n@htb[/htb]$ dig CH TXT version.bind 10.129.120.85

; <<>> DiG 9.10.6 <<>> CH TXT version.bind
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 47786
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; ANSWER SECTION:
version.bind.      0      CH      TXT      "9.10.6-P1"

;; ADDITIONAL SECTION:
version.bind.      0      CH      TXT      "9.10.6-P1-Debian"

;; Query time: 2 msec
;; SERVER: 10.129.120.85#53(10.129.120.85)
;; WHEN: Wed Jan 05 20:23:14 UTC 2023
;; MSG SIZE rcvd: 101
```

We can use the option **ANY** to view all available records. This will cause the server to show us all available entries that it is willing to disclose. It is important to note that not all entries from the zones will be shown.

DIG - ANY Query

DIG - ANY Query

```
dbcypb0n@htb[/htb]$ dig any inlanefreight.htb @10.129.14.128

; <<>> DiG 9.16.1-Ubuntu <<>> any inlanefreight.htb @10.129.14.128
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7649
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 5, AUTHORITY: 0, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 064b7e1f091b95120100000061476865a6026d01f87d10ca (good)
;; QUESTION SECTION:
inlanefreight.htb.      IN      ANY

;; ANSWER SECTION:
inlanefreight.htb.      604800 IN      TXT      "v=spf1 include:mailgun.org include:_spf.google.com include:spf.p
inlanefreight.htb.      604800 IN      TXT      "atlassian-domain-verification=t1rKC68JFszSdCKVpw64A1QksWdXuYFUe
inlanefreight.htb.      604800 IN      TXT      "MS=ms97310371"
inlanefreight.htb.      604800 IN      SOA      inlanefreight.htb. root.inlanefreight.htb. 2 604800 86400 2419200
inlanefreight.htb.      604800 IN      NS       ns.inlanefreight.htb.

;; ADDITIONAL SECTION:
ns.inlanefreight.htb.   604800 IN      A        10.129.34.136

;; Query time: 0 msec
;; SERVER: 10.129.14.128#53(10.129.14.128)
;; WHEN: So Sep 19 18:42:13 CEST 2021
;; MSG SIZE rcvd: 437
```

Zone transfer refers to the transfer of zones to another server in DNS, which generally happens over TCP port 53. This procedure is abbreviated **Asynchronous Full Transfer Zone (AXFR)**. Since a DNS failure usually has severe consequences for a company, the zone file is almost invariably kept identical on several name servers. When changes are made, it must be ensured that all servers have the same data. Synchronization between the servers involved is realized by zone transfer. Using a secret key **rndc-key**, which we have seen initially in the default configuration, the servers make sure that they communicate with their own master or slave. Zone transfer involves the mere transfer of files or records and the detection of discrepancies in the data sets of the servers involved.

The original data of a zone is located on a DNS server, which is called the **primary** name server for this zone. However, to increase the reliability, realize a simple load distribution, or protect the primary from attacks, one or more additional servers are installed in practice in almost all cases, which are called **secondary** name servers for this zone. For some **Top-Level Domains (TLDs)**, making zone files for the **Second Level Domains** accessible on at least two servers is mandatory.

DNS entries are generally only created, modified, or deleted on the primary. This can be done by manually editing the relevant zone file or automatically by a dynamic update from a database. A DNS server that serves as a direct source for synchronizing a zone file is called a master. A DNS server that obtains zone data from a master is called a slave. A primary is always a master, while a secondary can be both a slave and a master.

The slave fetches the **SOA** record of the relevant zone from the master at certain intervals, the so-called refresh time, usually one hour, and compares the serial numbers. If the serial number of the SOA record of the master is greater than that of the slave, the data sets no longer match.

DIG - AXFR Zone Transfer

DIG - AXFR Zone Transfer

```
dbcyp0n@htb[/htb]$ dig axfr inlanefreight.htb @10.129.14.128

; <<>> DiG 9.16.1-Ubuntu <<>> axfr inlanefreight.htb @10.129.14.128
;; global options: +cmd
inlanefreight.htb.      604800 IN      SOA      inlanefreight.htb. root.inlanefreight.htb. 2 604800 86400 2419200
inlanefreight.htb.      604800 IN      TXT      "MS=ms97310371"
inlanefreight.htb.      604800 IN      TXT      "atlassian-domain-verification=t1rKCy68JFszSdCKVpw64A1QksWdXuYFUe
inlanefreight.htb.      604800 IN      TXT      "v=spf1 include:mailgun.org include:_spf.google.com include:spf.p
inlanefreight.htb.      604800 IN      NS       ns.inlanefreight.htb.
app.inlanefreight.htb.  604800 IN      A        10.129.18.15
internal.inlanefreight.htb. 604800 IN      A        10.129.1.6
mail1.inlanefreight.htb. 604800 IN      A        10.129.18.201
ns.inlanefreight.htb.   604800 IN      A        10.129.34.136
inlanefreight.htb.      604800 IN      SOA      inlanefreight.htb. root.inlanefreight.htb. 2 604800 86400 2419200
;; Query time: 4 msec
;; SERVER: 10.129.14.128#53(10.129.14.128)
;; WHEN: So Sep 19 18:51:19 CEST 2021
;; XFR size: 9 records (messages 1, bytes 520)
```

If the administrator used a subnet for the **allow-transfer** option for testing purposes or as a workaround solution or set it to **any**, everyone would query the entire zone file at the DNS server. In addition, other zones can be queried, which may even show internal IP addresses and hostnames.

DIG - AXFR Zone Transfer - Internal

DIG - AXFR Zone Transfer - Internal


```

dbcyp0n@htb[/htb]$ dig axfr internal.inlanefreight.htb @10.129.14.128

; <<> DiG 9.16.1-Ubuntu <<> axfr internal.inlanefreight.htb @10.129.14.128
;; global options: +cmd
internal.inlanefreight.htb. 604800 IN      SOA      inlanefreight.htb. root.inlanefreight.htb. 2 604800 86400 2419200
internal.inlanefreight.htb. 604800 IN      TXT      "MS=ms97310371"
internal.inlanefreight.htb. 604800 IN      TXT      "atlassian-domain-verification=t1rKCy68JFszSdCKVpw64A1QksWdXuYFUe
internal.inlanefreight.htb. 604800 IN      TXT      "v=spf1 include:mailgun.org include:_spf.google.com include:spf.p
internal.inlanefreight.htb. 604800 IN      NS       ns.inlanefreight.htb.
dc1.internal.inlanefreight.htb. 604800 IN A      10.129.34.16
dc2.internal.inlanefreight.htb. 604800 IN A      10.129.34.11
mail1.internal.inlanefreight.htb. 604800 IN A      10.129.18.200
ns.internal.inlanefreight.htb. 604800 IN A      10.129.34.136
vpn.internal.inlanefreight.htb. 604800 IN A      10.129.1.6
ws1.internal.inlanefreight.htb. 604800 IN A      10.129.1.34
ws2.internal.inlanefreight.htb. 604800 IN A      10.129.1.35
wsus.internal.inlanefreight.htb. 604800 IN A      10.129.18.2
internal.inlanefreight.htb. 604800 IN      SOA      inlanefreight.htb. root.inlanefreight.htb. 2 604800 86400 2419200
;; Query time: 0 msec
;; SERVER: 10.129.14.128#53(10.129.14.128)
;; WHEN: So Sep 19 18:53:11 CEST 2021
;; XFR size: 15 records (messages 1, bytes 664)

```

The individual **A** records with the hostnames can also be found out with the help of a brute-force attack. To do this, we need a list of possible hostnames, which we use to send the requests in order. Such lists are provided, for example, by [SecLists](#).

An option would be to execute a **for-loop** in Bash that lists these entries and sends the corresponding query to the desired DNS server.

Subdomain Brute Forcing

Subdomain Brute Forcing

```

dbcyp0n@htb[/htb]$ for sub in $(cat /opt/useful/SecLists/Discovery/DNS/subdomains-top1million-110000.txt);do dig
ns.inlanefreight.htb. 604800 IN      A      10.129.34.136
mail1.inlanefreight.htb. 604800 IN      A      10.129.18.201
app.inlanefreight.htb. 604800 IN      A      10.129.18.15

```

Many different tools can be used for this, and most of them work in the same way. One of these tools is, for example [DNSenum](#).

Subdomain Brute Forcing

```
dbcyph0n@htb[/htb]$ dnsenum --dnsserver 10.129.14.128 --enum -p 0 -s 0 -o subdomains.txt -f /opt/useful/SecLists/
dnsenum VERSION:1.2.6

-----   inlanefreight.htb   -----

Host's addresses:
-----

Name Servers:
-----

ns.inlanefreight.htb.                604800   IN      A       10.129.34.136

Mail (MX) Servers:
-----

Trying Zone Transfers and getting Bind Versions:
-----

unresolvable name: ns.inlanefreight.htb at /usr/bin/dnsenum line 900 thread 1.

Trying Zone Transfer for inlanefreight.htb on ns.inlanefreight.htb ...
AXFR record query failed: no nameservers

Brute forcing with /home/cry0l1t3/Pentesting/SecLists/Discovery/DNS/subdomains-top1million-110000.txt:
-----

ns.inlanefreight.htb.                604800   IN      A       10.129.34.136
mail1.inlanefreight.htb.            604800   IN      A       10.129.18.201
app.inlanefreight.htb.              604800   IN      A       10.129.18.15
ns.inlanefreight.htb.                604800   IN      A       10.129.34.136

...SNIP...
done.
```

VPN Servers

 Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

☒ UDP 1337 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE

Start Instance

∞ / 1 spawns left

Waiting to start..

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)

 Cheat Sheet

 Download VPN Connection File

+ 1  Interact with the target DNS using its IP address and enumerate the FQDN of it for the "inlane freight.htb" domain.

Submit your answer here...

 Submit

+ 1  Identify if its possible to perform a zone transfer and submit the TXT record as the answer. (Format: HTB{...})

Submit your answer here...

 Submit

 Hint

+ 1  What is the IPv4 address of the hostname DC1?

Submit your answer here...

 Submit

+ 1  What is the FQDN of the host where the last octet ends with "x.x.x.203"?

Submit your answer here...

Cheat Sheet
Resources
? Go to Questions

Table of Contents

Introduction

Enumeration Principles	✓
Enumeration Methodology	✓

Infrastructure Based Enumeration

Domain Information	✓
Cloud Resources	✓
Staff	✓

Host Based Enumeration

FTP	✓
SMB	✓
NFS	✓
DNS	
SMTP	
IMAP / POP3	
SNMP	
MySQL	
MSSQL	
Oracle TNS	
IPMI	

Remote Management Protocols

Linux Remote Management Protocols	
Windows Remote Management Protocols	

Skills Assessment

Footprinting Lab - Easy	
Footprinting Lab - Medium	
Footprinting Lab - Hard	

My Workstation

OFFLINE

 Start Instance

 / 1 spawns left